

Ânalyse dos algoritmos de ordenação clássicos

Nattan Ferreira

Junho 2024

NOME:Nattan Ferreira da Silva

NUSP:15520641

TURMA : primeiro semestre BCC

Introdução

Atualmente, é fato que existem diversos programas super eficiente e Inteligências artificias que realizam processos extraordinários e as pessoas cada vez mais ficam surpresas com a capacidade de evolução de um computador e, ao mesmo tempo, assustadas com esta, porém, mal elas sabem que por trás desses super programas e super IAs existem algoritmos que calculam e ordenam centenas de números , levando um tempo considerável para fazer tais operações. Por isso, vamos analisar alguns algoritmos clássicos de ordenação crescente como: Bubble Sort , Counting Sort , Selection Sort , Insertion Sort e o algoritmo de ordenação nativo do Python(Timsort), de modo que tenhamos noção do tempo necessário gasto em cada algoritmo para ordenar uma lista com n elementos.

Materiais e métodos

Como citado anteriormente iremos analisar cientificamente alguns algoritmos clássicos, portanto, primeiramente, vamos explicar como cada um deles funciona.

Bubble Sort:

Bubble Sort, ou, se preferir, ordenação de bolhas, é um algoritmo que faz jus ao seu nome. Dada uma lista de números desordenada, seu funcionamento consiste em comparar repetidamente pares de números vizinhos na lista, trocando-os de posição se for preciso, para garantir que o maior número avance para a direita. Por exemplo, dados os números x e y , se $x > y$, então x será colocado à direita de y ; senão, nada acontece. Tal processo é repetido durante toda a lista e, por conseguinte, o maior elemento da lista é levado como uma bolha ao final da lista, já que a comparação é feita com todos os números. Para achar o segundo maior número, o processo é feito uma segunda vez, excluindo o número já ordenado; para o terceiro maior número, uma terceira vez, e assim por diante, até a lista estar completamente ordenada.

Selection Sort

Selection Sort, possui algumas diferenças em relação ao Bubble Sort. Primeiramente, definimos o primeiro elemento da lista como sendo o menor, e percorremos a partir do segundo elemento comparando cada elemento com o menor elemento atual, se o elemento comparado for menor que o menor elemento atual, definimos este como menor elemento atual, e segue nesse processo até o final da lista, assim que terminarmos, comparamos o menor elemento atual com o menor que definimos anteriormente, se este for maior que o menor elemento atual então, trocamos-los de posição, senão, o elemento definido anteriormente como menor está ordenado, logo, partimos para o segundo elemento da lista para fazermos o mesmo processo, e assim por diante até a lista estar ordenada.

Insertion Sort

Insertion Sort, como seu nome diz, irá inserir os números em suas posições corretas, é abstrato, mas irei apresentar o resumo do funcionamento desse algoritmo. Para cada elemento da lista, irá ser feita comparações entre dois números x e y se $x < y$, então x e y estão ordenados, logo, o algoritmo continuará fazendo as comparações com os próximos números da lista e se, em algum momento, algum $x > y$, então, y trocará de lugar com x e, a partir desse momento será necessário fazer as comparações de y com os

números que estão à esquerda de y , já que não sabemos se existe um elemento maior que y desse lado , se algum número do lado esquerdo de y não for maior que ele , então , a lista , nessa parte, está ordenada , e fazemos o mesmo processo para os números à frente de y .

Counting Sort

Counting Sort é um algoritmo que funciona de uma maneira peculiar , pois , diferente dos outros algoritmos citados , ele não faz nenhuma comparação , seu funcionamento consiste em criar um vetor frequência dos números que compõem a lista que queremos ordenar , onde a ordenação acontece no momento de colocarmos cada frequência de x no vetor frequência, pois colocamos cada frequência de cada número em seu respectivo lugar , por exemplo uma lista que tenha algum $x = 1$, com frequência 2, colocaremos a frequência na posição 1 do vetor frequência , e assim por diante para os demais x , logo depois, somamos as frequências vizinhas de modo à obter quantos elementos são menores ou iguais á aquele x . Após isso , criamos uma lista normal do mesmo tamanho que a original , e colocamos os números ,que representam cada frequência , dentro do vetor, por exemplo, posição 1 , será $x_1 = 1$, com frequência 2 , e $x_2 = 2$ com frequência 1, logo , como é feita a soma das frequências , existem 3 números menores ou iguais a x_2 , como é trivial enxergar , são esses x_1 e o próprio x_2 , logo x_2 será posicionado na posição 3 do vetor e ,na posição 1 e 2, será posicionado x_1 , já que x_1 é menor que x_2 .O processo é feito até o vetor frequência estar completamente vazio , ordenando ,assim , a lista.

Análise

Por fim , temos a análise feita em cada algoritmo, é importante destacar os métodos e materiais usados para fazer essa análise. Como queremos saber como cada algoritmo se comporta temporariamente , foram feitos testes em cada algoritmo usando listas com 1000,5000,10000, 50000 e 100000 números aleatórios , respectivamente .Vale ressaltar que , o teste foi feito na linguagem de programação Python e implementado em uma máquina com processador AMD Ryzen 5 4500H , 8gb de memória Ram e sistema operacional Debian Linux ,além disso , usamos a biblioteca Matplotlib para

criar gráficos de tempos médios demorados para cada algoritmo ordenar todas listas dos tamanhos citados. Os testes se resumem em cada algoritmo de ordenação utilizado, ordenando cada tamanho 10 vezes, colocamos o tempo levado para cada ordenação em uma lista z e, no final, calculamos a média de z , além disso, como há variações nos tempos, calculamos, também, a variância de z , tal processo é repetido para todos tamanhos utilizados e todos algoritmos, depois disso, utilizamos a biblioteca Matplotlib para criar um gráfico contendo os gráficos de tempo médio em função da quantidade de elementos na lista para cada algoritmo, utilizamos no eixo y do gráfico a lista de médias de cada algoritmo e, como ela não é precisa, utilizamos também uma correção calculando o desvio padrão usando a variância sobre a lista de tempos calculada antes, ademais, no eixo x utilizamos quantos elementos cada lista possui. Outrossim, analisamos também, como os tempos médios de ordenação dos algoritmos Bubble Sort, Timsort e Insertion Sort se comportam ao ordenar uma lista de 100 mil números com uma certo $p\%$ de desordenamento, o processo foi feito exatamente como o anterior, porém, invés de usarmos as listas de 1000, 5000, 10000, 50000 e 100000 números aleatórios, usaremos apenas uma de 100 mil números com apenas $p\%$ de desordenamento, os testes foram feitos para $p = 1$, $p = 3$, $p = 5$, $p = 10$ e $p = 50$, para cada algoritmo foi feito o mesmo processo de calcular 10 vezes o tempo e obter a média como na análise anterior, contudo, cada processo é feito para cada p e cada algoritmo e o resultado é utilizado para criar gráfico contendo os gráficos de tempo médio em função de $p\%$ de desordenamento da lista para cada algoritmo usado no teste.

Perguntas científicas

A primeira pergunta que devemos responder em nossa análise é de qual o tempo médio necessário para cada algoritmo ordenar listas de 1000, 5000, 10000, 50000 e 100000 números, para a responder iremos tomar como base o sistema de Big O Notation, que usa como referência como os algoritmos de ordenação se comportam, em seu pior caso, em função da quantidade de elementos. Idealmente, nessa representação temos padrões de complexidade para os algoritmos citados em seus piores e melhores casos, respectivamente:

Bubble sort :

Em seu pior caso o Bubble se comporta como $O(n^2)$, onde n é a quantidade de elementos na lista, podemos perceber usando o código a baixo como base, que na primeira repetição externa, será feito $n - 1$ comparações, na segunda $n - 2$ e assim por diante, logo no final temos uma quantidade de comparações igual à $n - 1 + n - 2 + n - 3 + \dots + 1$, até $n - (n - 1)$ que é última iteração, percebemos uma padrão de uma P.A de razão -1 , logo ao colocarmos na fórmula da soma de todos termos de uma P.A temos que $S = (n - 1 + 1)n / 2 = n^2 / 2$, ou seja, quadrático. Em seu melhor caso, é onde, não trocamos nenhum elemento, ou seja, a lista estando em ordem, logo, $O(n)$.

Listing 1: Código de Ordenação Bubble Sort

```
def bubble(V, n):
    for i in range(n):
        trocou = False
        for j in range(n - 1):
            if V[j] > V[j + 1]:
                trocou = True
                maior = V[j]
                V[j] = V[j + 1]
                V[j + 1] = maior
        if not trocou:
            break
```

Selection Sort :

Em seu pior caso possui complexidade $O(n^2)$, da mesma forma que o Bubble Sort, até as n interações são feitas $n - 1 + n - 2 + \dots + 1$ comparações, ou seja, $n^2 / 2$ comparações no total. Em seu pior caso, o mesmo fato se repete, logo, sua complexidade se mantém.

Listing 2: Código de Ordenação Selection Sort

```
def selection(V, n):
```

```

i = 0
while i < n:
    u = i + 1
    while u < n:
        menor = V[i]
        if V[i] > V[u]:
            V[i] = V[u]
            V[u] = menor
        u += 1
    i += 1

```

Insertion Sort :

em seu pior temos complexidade de $O(n^2)$, que é quando a lista está em ordem decrescente , já que cada interação de i será necessário uma comparação a mais que a anterior Por exemplo, na primeira interação terá uma comparação ,na segunda duas, ou seja ,

$$S = \sum_{k=1}^n k$$

, uma P.A de razão 1 onde S é o somatório de todas interações, colocando na formula de soma de todos termos temos $S = ((1 + n) * n) / 2 = n^2 + n / 2$, quadrático . Em seu melhor caso, temos que toda lista está ordenada, então, será necessário apenas uma interação pelo tamanho da lista, logo, $O(n)$.

Listing 3: Código de Ordenação Insertion Sort

```

def insertion(V,n):
    i = 0
    while i < n-1:
        menor = V[i+1]
        if menor < V[i]:
            V[i+1] = V[i]
            V[i] = menor
            menor = V[i]
            u = i-1
            while u >= 0:

```

```

        menor = V[u+1]
        if V[u] > menor:
            V[u+1] = V[u]
            V[u] = menor
            menor = V[u]
        u += -1
    i += 1

```

Counting Sort

Em ambos casos o Counting sort possui complexidade $O(n + k)$, já que existe apenas as interações na lista n que representa o vetor original e na lista k que representa o vetor frequência

Tim Sort

$O(n \log n)$ em seu pior caso , no melhor caso temos $O(n)$

Agora que sabemos como cada algoritmo se comporta teoricamente, iremos comparar esse comportamento citado em cada algoritmo com o obtido no gráfico referente a primeira pergunta.

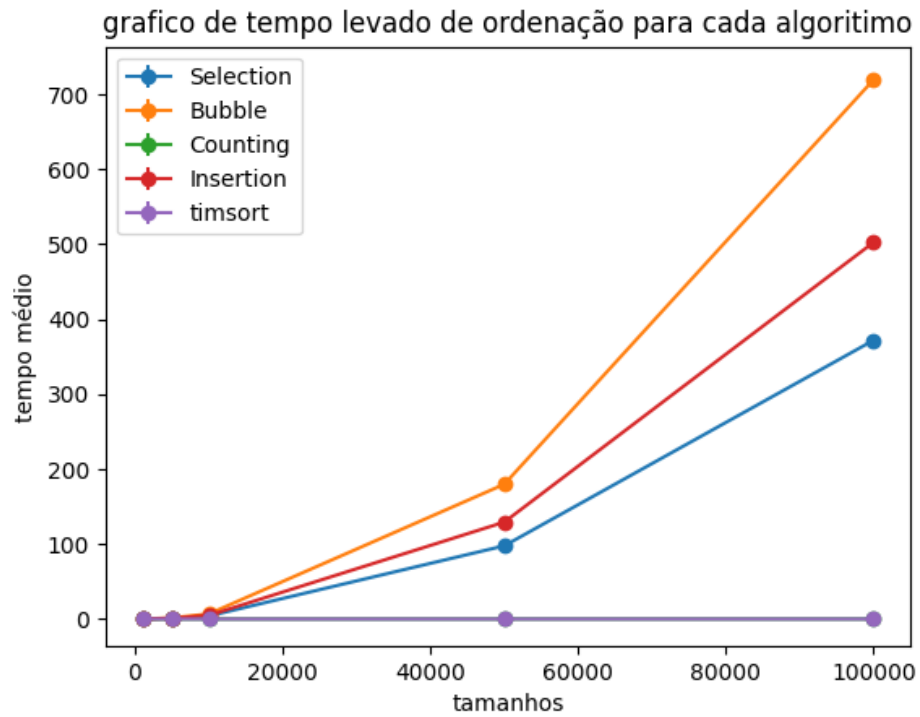


Figura 1: gráfico referente ao experimento 1

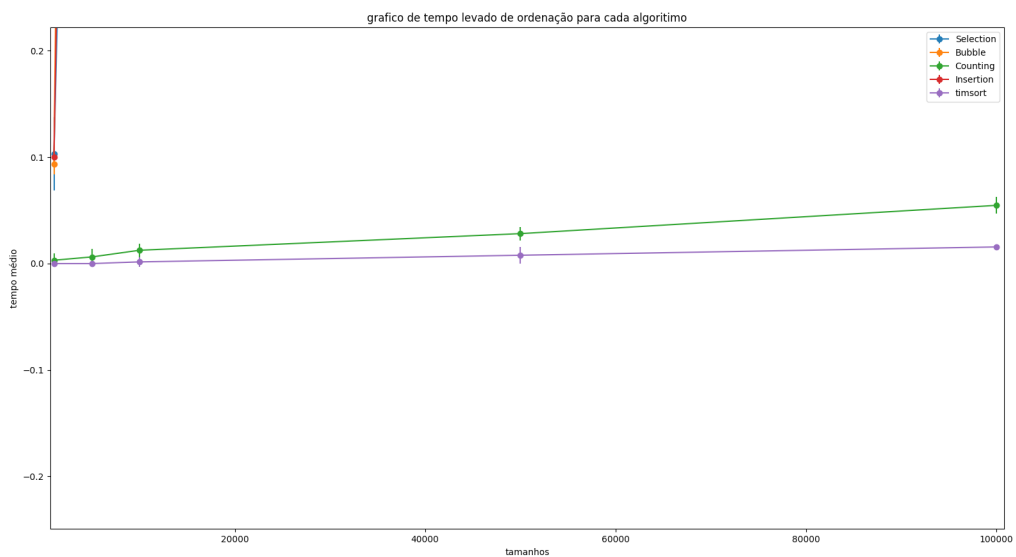


Figura 2: zoom no Counting Sort e Tim Sort

Elementos	Selection	Bubble	Counting	Insertion	Timsort
1 000	0.04	0.06	0.00135	0.04	6.61
5 000	1.02	1.63	0.0029	1.19	0.00042
10 000	4.019	6.81	0.0045	4.85	0.00093
50 000	97.69	179.83	0.019	129.32	0.006
100 000	371.97	719.64	0.043	502.65	0.01

Figura 3: tabela de tempo médio , aproximado , por quantidade de elementos para todo algoritmo.

Elementos	Selection	Bubble	Counting	Insertion	Timsort
1 000	0.0003	0.001	8.5	4.9	6.3
5 000	0.011	0.002	0.0003	4.3	6
10 000	0.014	0.03	9.5	0.0004	9.5
50 000	0.59	5.9	0.0005	2.4	0.00034
100 000	0.37	7.9	0.001	0.7	0.0006

Figura 4: tabela de desvio padrão , aproximado , por quantidade de elementos para todo algoritmo.

Bubble sort

Como destacado na Big O Notation , é notório , visualizando o gráfico , que quanto mais elementos presentes na lista mais o coeficiente angular referente ao gráfico do Bubble Sort cresce , se aproximando à uma função quadrática .

Insertion Sort

Similarmente ao Bubble , visualmente percebemos que quanto mais elementos presentes na lista mais o coeficiente angular do gráfico cresce , porém , mesmo seu crescimento se aproximando de uma função quadrática , continua sendo mais vantajoso que o algoritmo de bolhas . Além disso, percebe – se que o algoritmo está atuando em seu pior caso , já que no seu melhor , este se aproximaria de $O(n)$, ou seja , uma função linear.

Selection Sort

De mesmo modo que os algoritmos passados , o gráfico do Selection Sort se aproxima de uma função quadrática , porém , sendo mais vantajoso , em relação à média de tempo , do que o Bubble e o Insertion .

Counting Sort

Seguindo a big O Notation teríamos um comportamento de $O(n+k)$, ou seja , um gráfico de reta , visualmente, percebemos que tal fato se confirma.

Timsort

Percebemos uma reta , ou seja , $O(n)$.

desfecho

Dada a análise feita , podemos confirmar que os algoritmos testados seguem suas respectivas classificações no sistema de Big O Notation .Ademais , podemos enxergar que mesmo a condição teórica da Big O Notation classificando um padrão de complexidade comportamental dos algoritmos , na prática , estes se diferem , sendo um mais vantajoso que outro , mesmo que a diferença, no final , seja imperceptível .

Segundo experimento

grafico de tempo levado de ordenação para cada algoritmo com p% desordenação

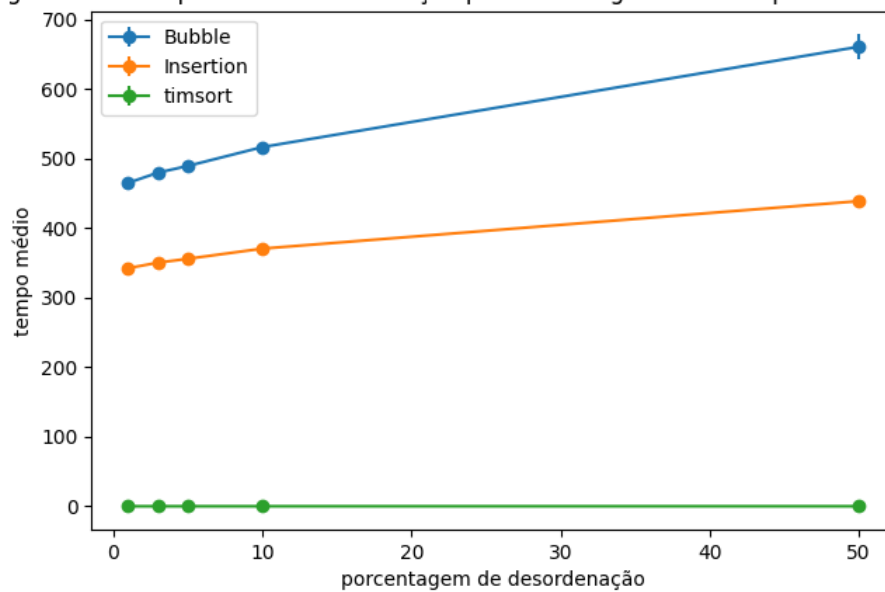


Figura 5: Gráfico referente ao experimento 2

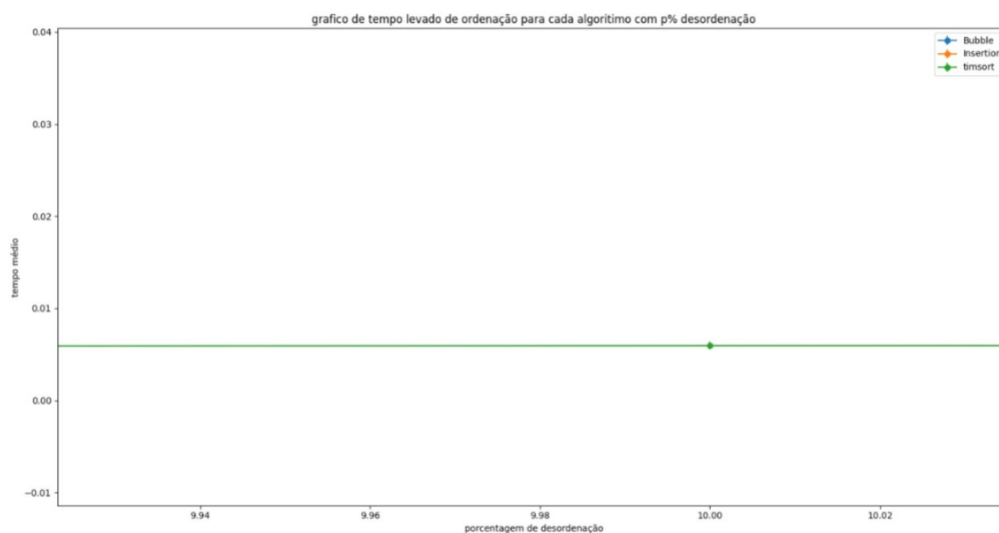


Figura 6: Zoom no Timsort

embaralhamento em %	Bubble	Insertion	Timsort
1	466.57	343.34	0.002
3	486.66	349.24	0.003
5	495.67	358.39	0.0041
10	513.54	370.75	0.006
50	690.60	432.30	0.01

Figura 7: tabela tempo médio, aproximado, por % de embaralhamento para todo algoritmo.

embaralhamento em %	Bubble	Insertion	Timsort
1	6.81	1.69	0.0003
3	7.56	0.39	0.0003
5	5.39	2.70	0.00032
10	1.96	0.30	0.00073
50	1.39	0.49	0.00075

Figura 8: tabela desvio padrão, aproximado por % de embaralhamento para todo algoritmo.

Para a segunda pergunta, temos o teste que revela qual é o tempo médio levado para os algoritmos Bubble Sort, Insertion Sort e Timsort ordenarem uma lista de 100 mil números com 1% 3% 5% 10% e 50% de desordenamento, o gráfico obtido pode ser visto logo acima.

Bubble Sort

diferente do teste anterior, como podemos perceber, o algoritmo se comportou em seu melhor caso, linearmente, ou seja, seguindo a Big O Notation, $O(n)$.

Insertion Sort

O mesmo ocorreu com o Insertion Sort que se comportou de forma linear, ou seja, em seu melhor caso

Timsort

Já o Timsort, aparentemente se apresenta como uma função constante, contudo, se analisarmos sua tabela acima, percebemos

que ocorre uma variação nos valores, logo , sua função se aproxima de uma função Afim.

Desfecho

Portanto , é claro como os algoritmos testados mudam sua complexidade ao colocarmos apenas uma porcentagem p de embaralhamento na lista , passando de sua complexidade no pior caso , para a sua no melhor caso, com exceção do Timsort . Ademais , como destacado no experimento anterior , mesmo com a mesma complexidade , os algoritmos tendem a ter vantagem sobre o outro .

Conclusão

Para concluir , com todo esse processo , chegamos no entendimento da complexidade, de uma das bases que compõe os programas e modelos de IAs atuais , ou seja, entendemos o quão duradouro pode ser para um algoritmo de ordenação realizar todas suas operações sobre uma determinada lista de números, os quais também variam de complexidade caso uma lista esteja quase ordenada , como visto no experimento. Tal fato nos ajuda a entender que todo esse processo de evolução de tecnológica não é apenas uma mágica , mas sim, milhões de operações e ordenações de números. Ademais, esse foi um experimento que finalmente consegui entender a tão necessária Big O Notation , também pude entender que mesmo algoritmos que possuem a mesma complexidade podem se diferir em relação ao tempo necessário para fazer sua devida operação . Um dos grandes problemas que enfrentei foi a gigante quantidade de números na lista , a qual tornou o processo de ordenação um verdadeiro desafio para os algoritmos ,levando até mesmo dias para completar a ordenação , tal fato me fez passar por diversas dificuldades , já que era impossível saber se o experimento ainda estava funcionando ou não. Por fim , tirando essas complicações, foi divertido escrever os códigos desses algoritmos clássicos e enxergar suas complexidades através de gráficos .